Portfolio Laravel 13 · CRUD, relaciones y datos de prueba

Alumno/a: Cristina **Proyecto:** gamemanager **Modalidad:** B **Fecha:** 2026-11-09

Ampliación con relación 1:N

En mi proyecto GameManager, una Desarrolladora tiene muchos Videojuegos y cada Videojuego pertenece a una Desarrolladora. Esta relación 1:N permite estructurar el catálogo asociando la autoría corporativa a cada título, facilitando el filtrado por estudio creador y evitando la redundancia de datos de la empresa.

Pruebas realizadas: He generado datos iniciales mediante factories y seeders con relaciones anidadas. He comprobado en el navegador el listado principal cargando la relación con Videojuego::with(['desarrolladora', 'generos']) para evitar el problema N+1. Además, verifiqué que la eliminación de un juego no borra a su desarrolladora y que una desarrolladora con videojuegos no puede ser eliminada gracias al control del controlador y restrictOnDelete().

Autoevaluación

1. ¿Qué comando genera modelo, migración y Resource Controller? — **php artisan make:model Curso -mcr** (correcto)
2. ¿Qué elemento permite la asignación masiva mediante create()? — **$fillable** (correcto)
3. ¿Qué registra automáticamente las siete rutas REST? — **Route::resource** (correcto)
4. ¿Qué directiva protege los formularios POST frente a CSRF? — **@csrf** (correcto)
5. ¿Cómo se simula PUT desde un formulario HTML POST? — **@method('PUT')** (correcto)
6. ¿Qué directiva reutiliza un fragmento Blade? — **@include** (correcto)
7. ¿Qué facilita Route Model Binding? — **Convertir el parámetro de ruta en un modelo Eloquent** (correcto)

8. ¿Qué método del Form Request proporciona los datos validados? — **validated()** (correcto)
9. ¿Qué regla permite editar un valor único propio sin marcarlo como duplicado? — **ignore($modelo) sobre Rule::unique** (correcto)
10. ¿Qué relación define que un curso tiene muchas lecciones? — **hasMany()** (correcto)
11. En el modelo Leccion, ¿qué relación apunta al curso padre? — **belongsTo()** (correcto)
12. ¿Cómo evitar consultas adicionales para obtener las lecciones de muchos cursos? — **Curso::with('lecciones')->get()** (correcto)
13. ¿Qué opción define curso_id como clave foránea? — **$table->foreignId('curso_id')->constrained('cursos')** (correcto)
14. ¿En qué método de una factory se definen los atributos falsos? — **definition()** (correcto)
15. ¿Qué comando ejecuta los seeders? — **php artisan db:seed** (correcto)

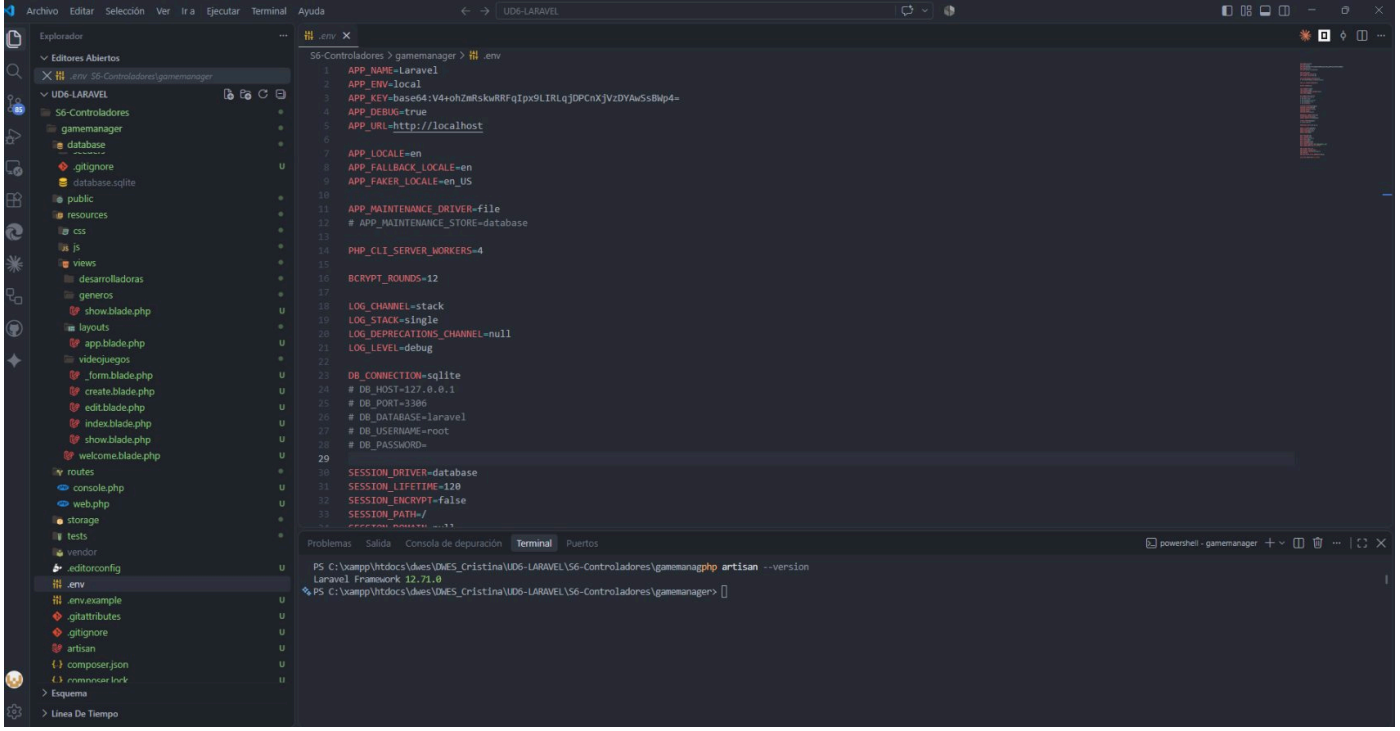
Evidencias

1. Proyecto y entorno

Muestra la carpeta del proyecto, la versión de Laravel y su conexión a la base de datos (sin mostrar credenciales).

Palabra clave: php artisan --version

Comentario: Muestra la terminal en la raíz del proyecto ejecutando Laravel 12 y el archivo .env configurado con SQLite como motor de base de datos local para la persistencia del proyecto

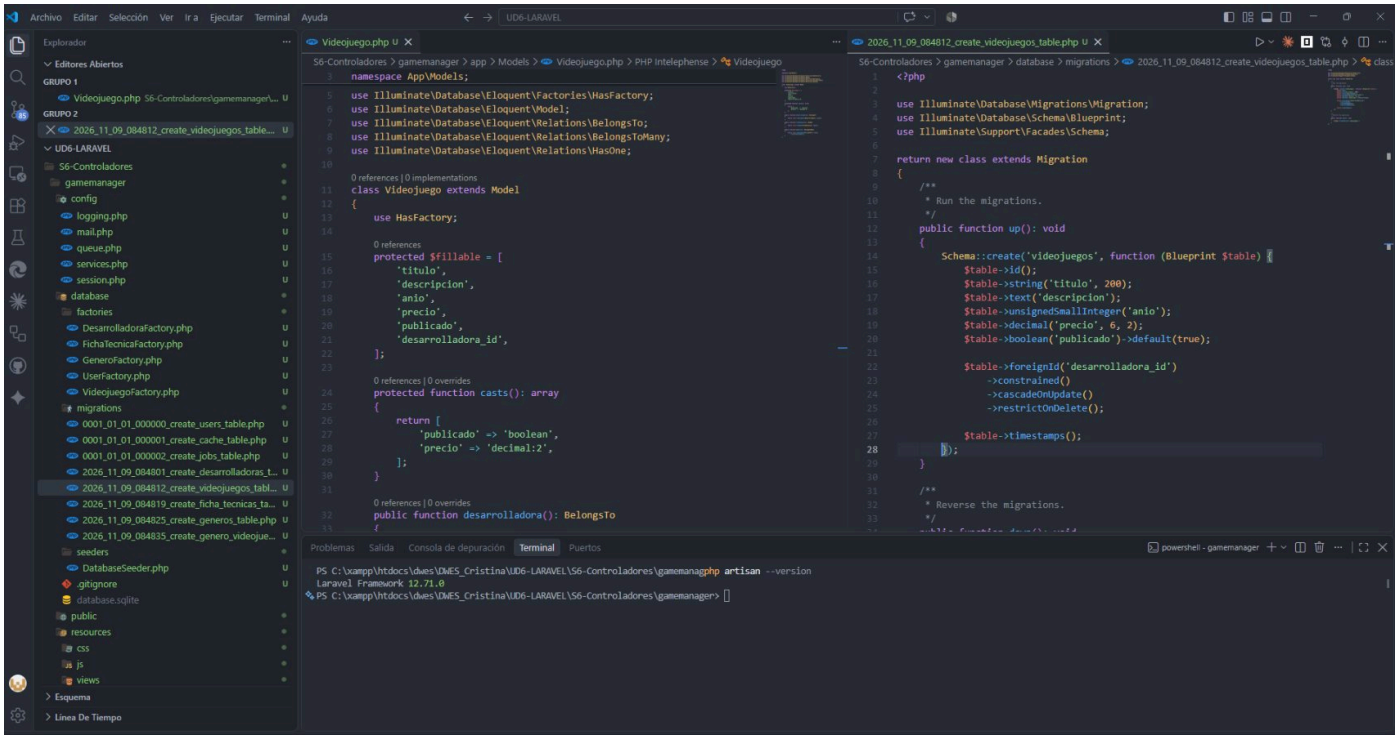


2. Modelo y migración principal

Captura el modelo principal y la migración con seis campos de negocio, clave primaria y timestamps.

Palabra clave: php artisan make:model Curso -mcr

Comentario: Presenta el modelo principal Videojuego y su migración correspondiente con sus campos de negocio, tipos de datos, timestamps y la clave foránea vinculada a desarrolladoras



3. Campos, fillable y casts

Señala $fillable, casts y qué columnas admiten asignación masiva.

Palabra clave: protected $fillable

Comentario: Demuestra la configuración de asignación masiva mediante $fillable y la conversión automática de tipos en casts() para el booleano publicado y el formato decimal del precio



4. Migraciones ejecutadas

Muestra el estado de las migraciones y las tablas creadas.

Palabra clave: php artisan migrate:status

Comentario: Comprueba el estado de la base de datos tras ejecutar las migraciones, verificando que las tablas de desarrolladoras, videojuegos, fichas técnicas, géneros y la pivot están creadas correctamente



5. Rutas REST registradas

Captura routes/web.php y/o route:list con los siete nombres de rutas.

Palabra clave: Route::resource

Comentario: Muestra la definición del recurso REST en web.php y el listado de las 7 rutas generadas con sus métodos HTTP, URIs, nombres de ruta y acciones asociadas del controlador

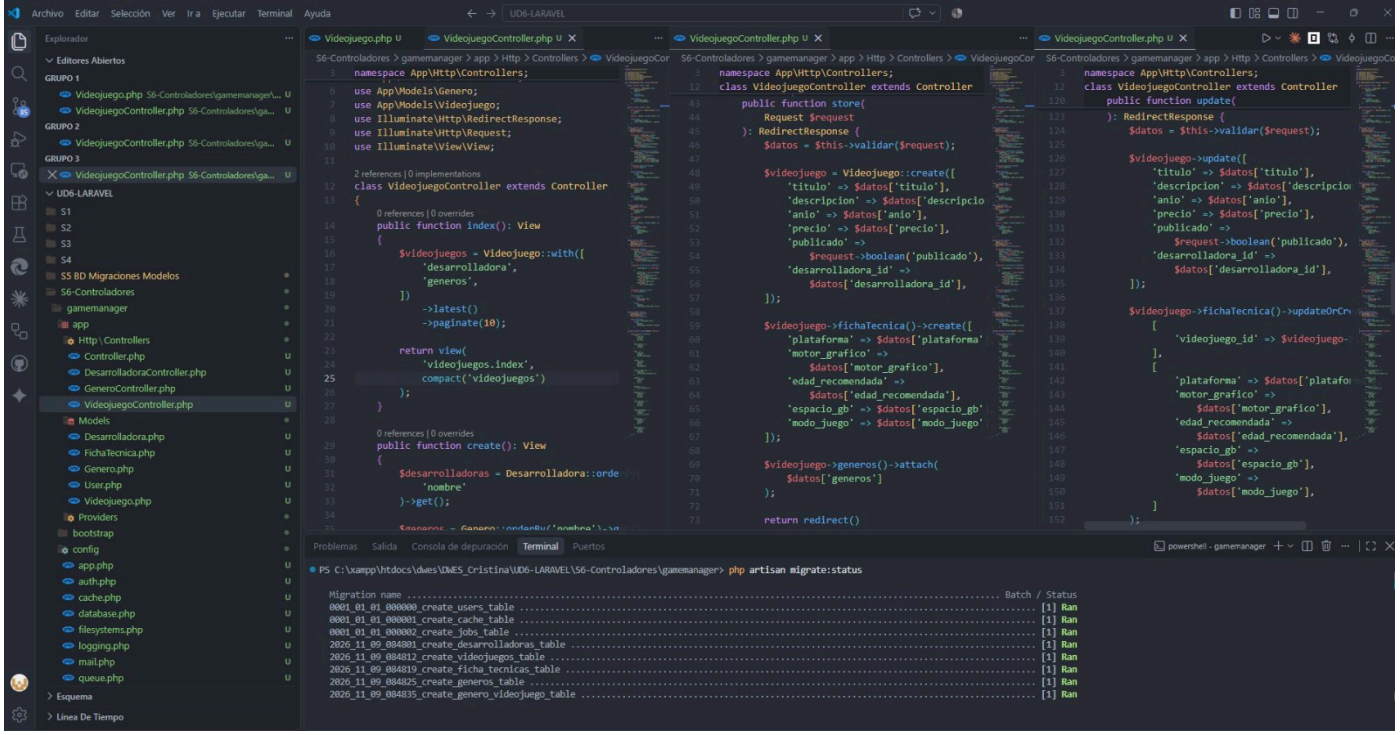


6. Resource Controller

Captura los siete métodos y destaca index(), store() y update().

Palabra clave: index / create / store / show / edit / update / destroy

Comentario: Código de VideojuegoController estructurado con los métodos estándar del recurso, gestionando la inserción de registros principales y la delegación de datos hacia las relaciones

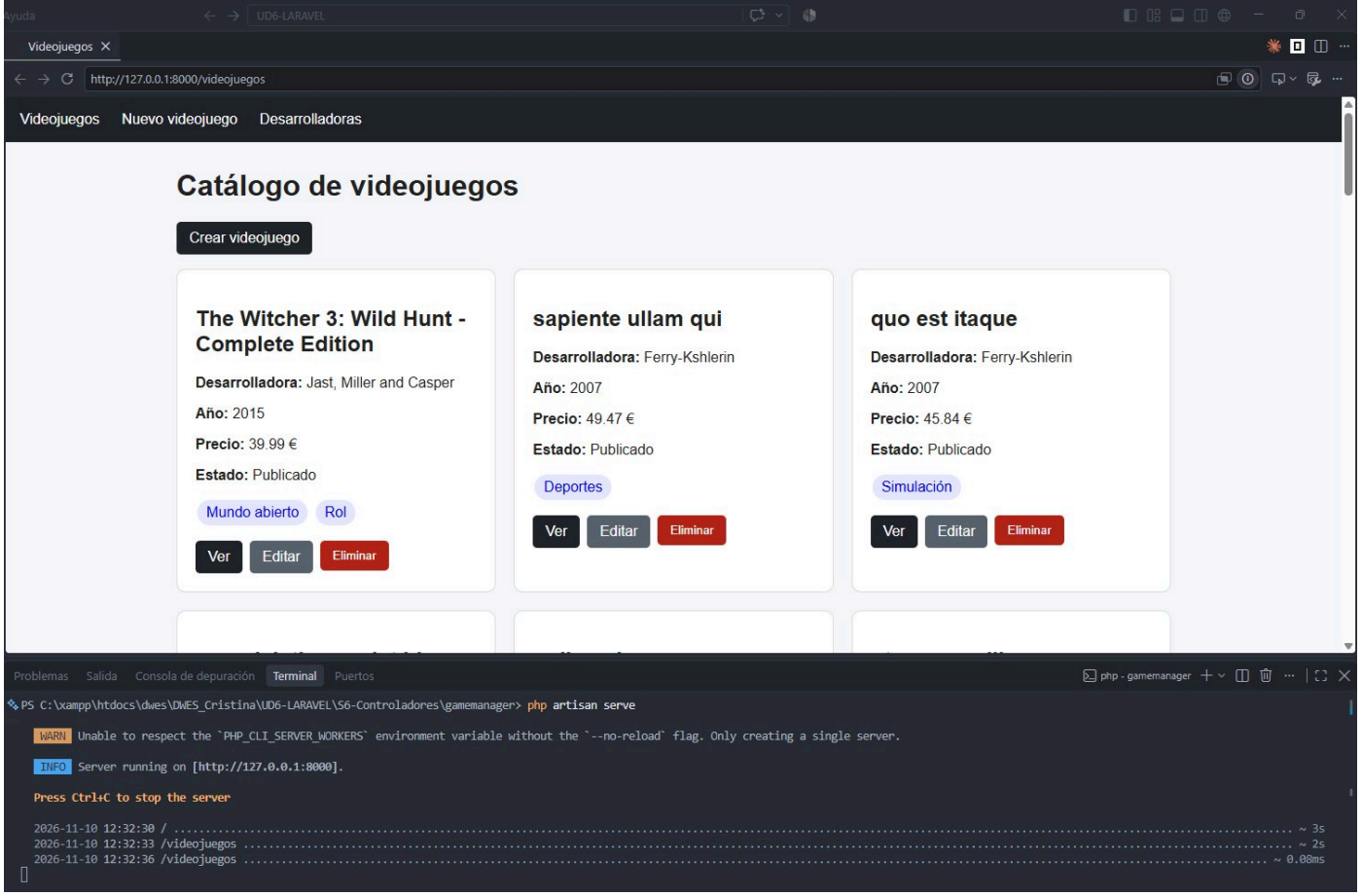


7. Listado de registros

Muestra la interfaz funcionando con al menos tres registros.

Palabra clave: @forelse

Comentario: Interfaz gráfica del catálogo en el navegador mostrando varios videojuegos generados por los seeders con su título, precio, desarrolladora, etiquetas de género y acciones

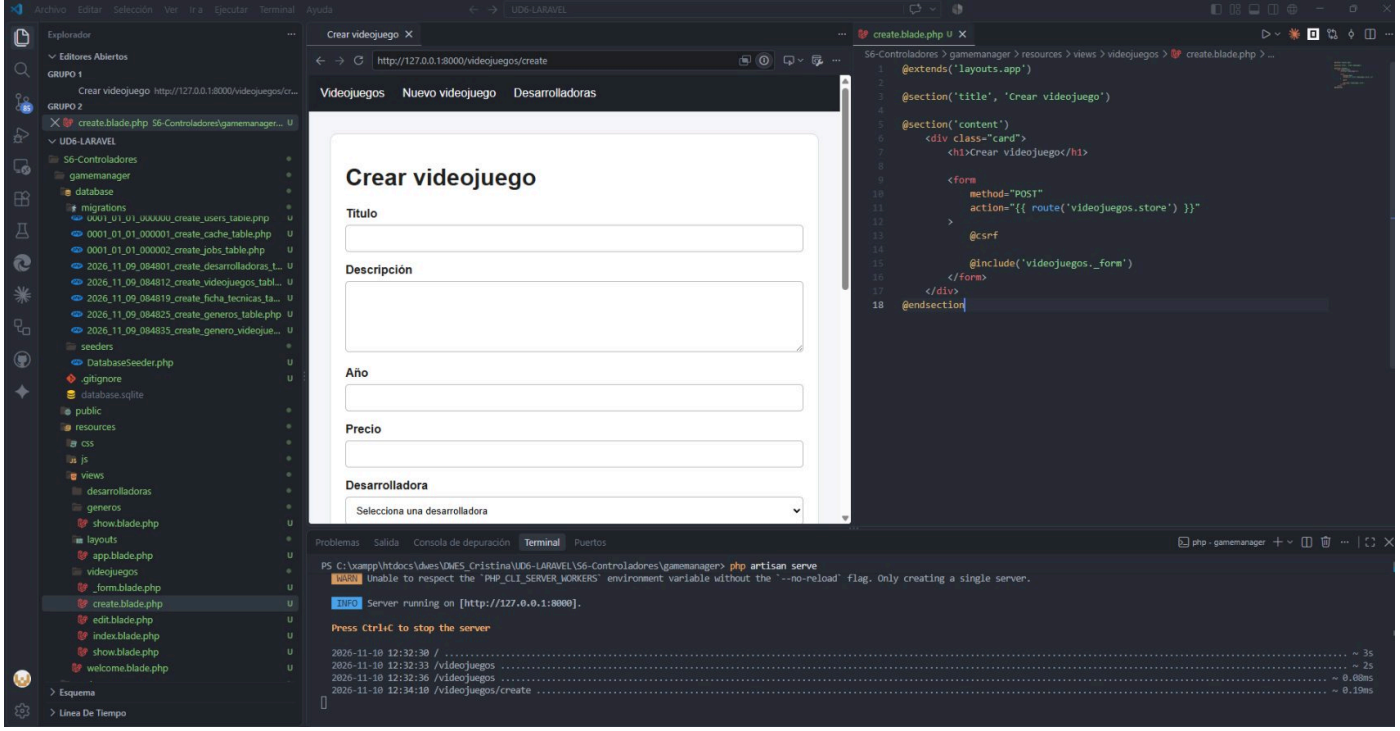


8. Alta y protección CSRF

Captura el formulario create y @csrf; muestra un alta satisfactoria.

Palabra clave: @csrf

Comentario: Formulario de alta de videojuegos en funcionamiento y el código Blade que implementa la directiva @csrf para generar el token de protección contra ataques Cross-Site Request Forgery

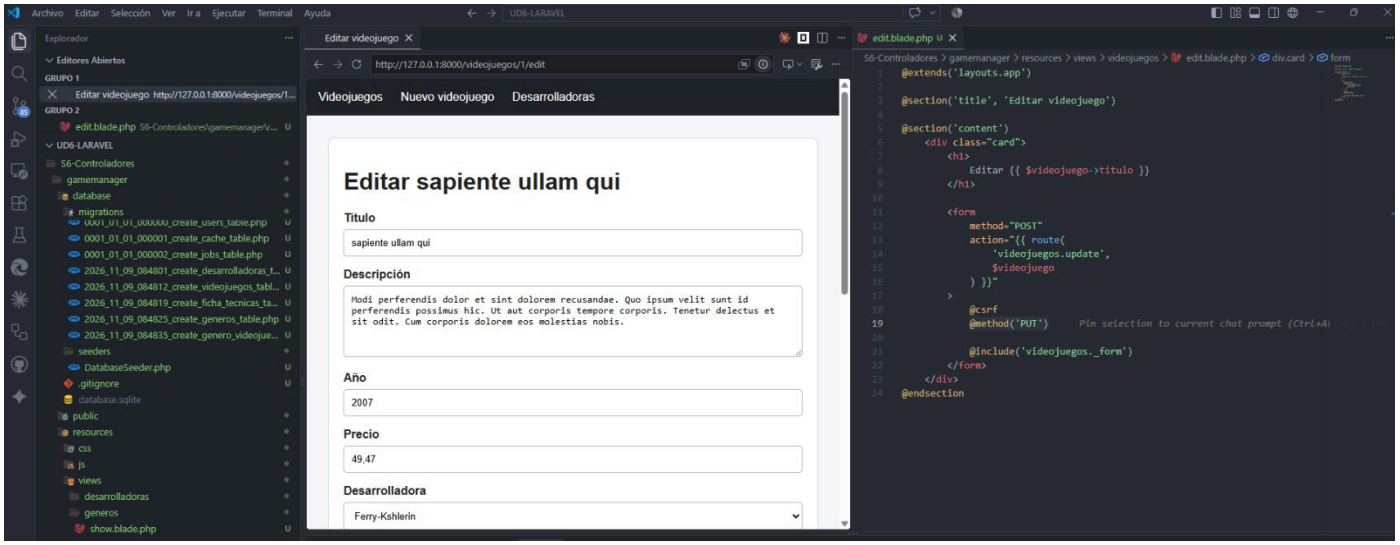


9. Edición con PUT

Muestra el formulario edit, los datos precargados y el resultado de editar.

Palabra clave: @method('PUT')

Comentario: Vista de edición cargando los valores actuales mediante old() y la directiva Blade @method('PUT') que permite la simulación del método HTTP PUT en el formulario HTML

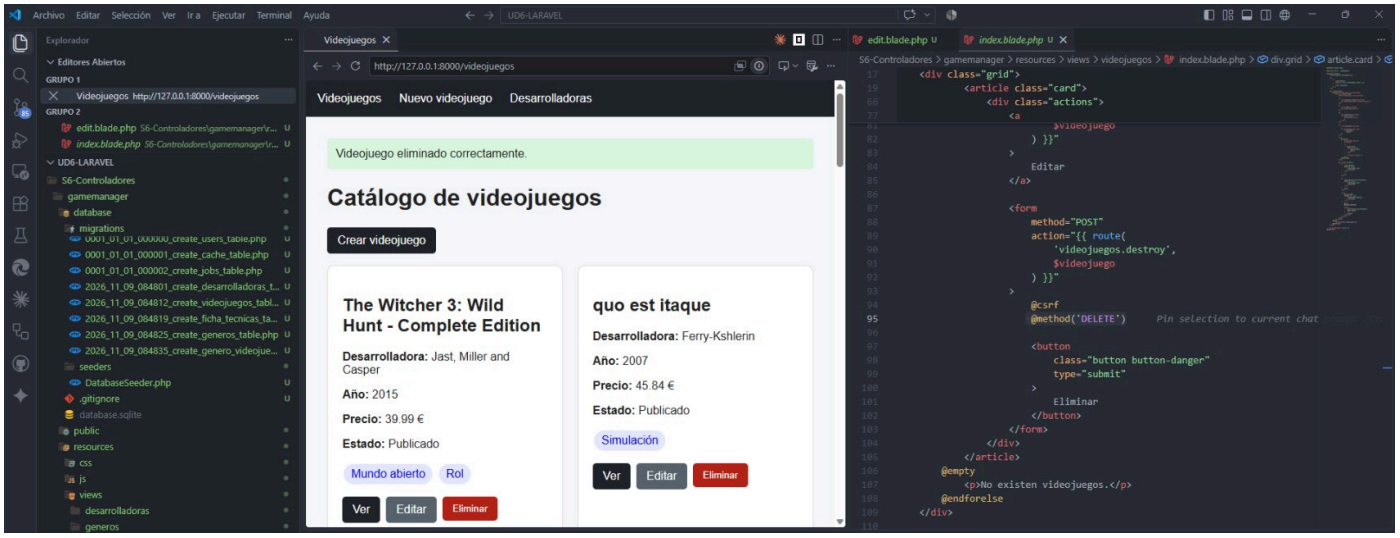


10. Eliminación con DELETE

Muestra el código del formulario de eliminación y el registro eliminado.

Palabra clave: @method('DELETE')

Comentario: Demuestra el formulario de borrado configurado con el método HTTP DELETE y la notificación de sesión confirmando la eliminación correcta del videojuego del catálogo

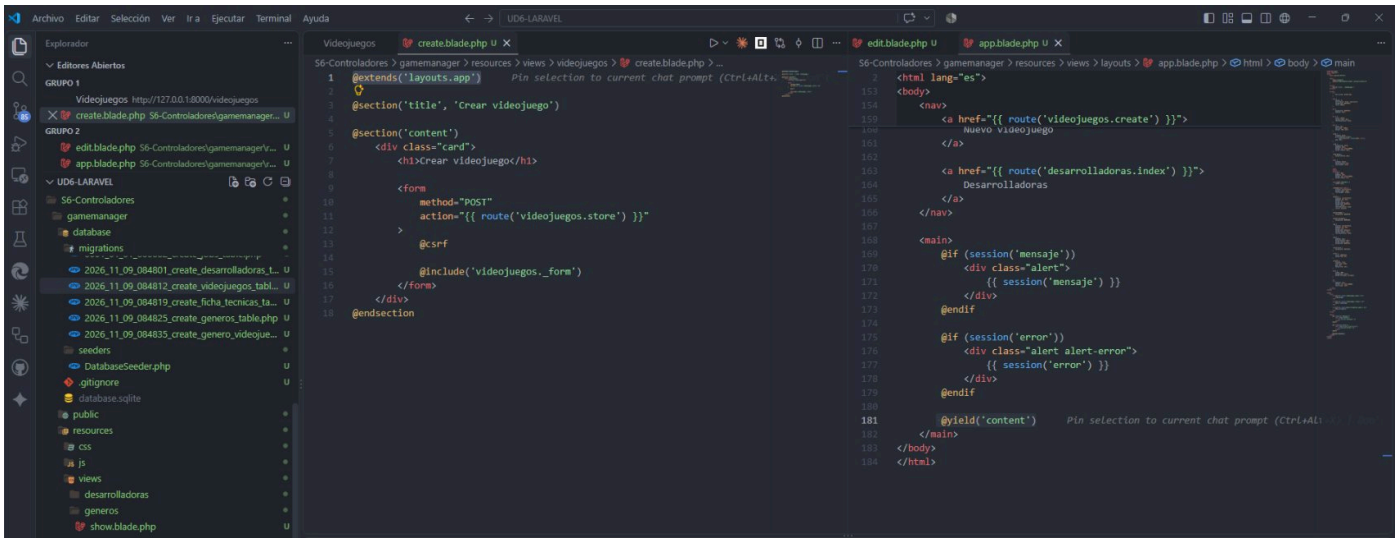


11. Layout y partial

Captura el layout Blade, extends/yield y formulario parcial incluido.

Palabra clave: @include

Comentario: Estructura modular de vistas Blade mediante un layout base compartido con @yield y la inclusión del subformulario reutilizable _form.blade.php mediante @include

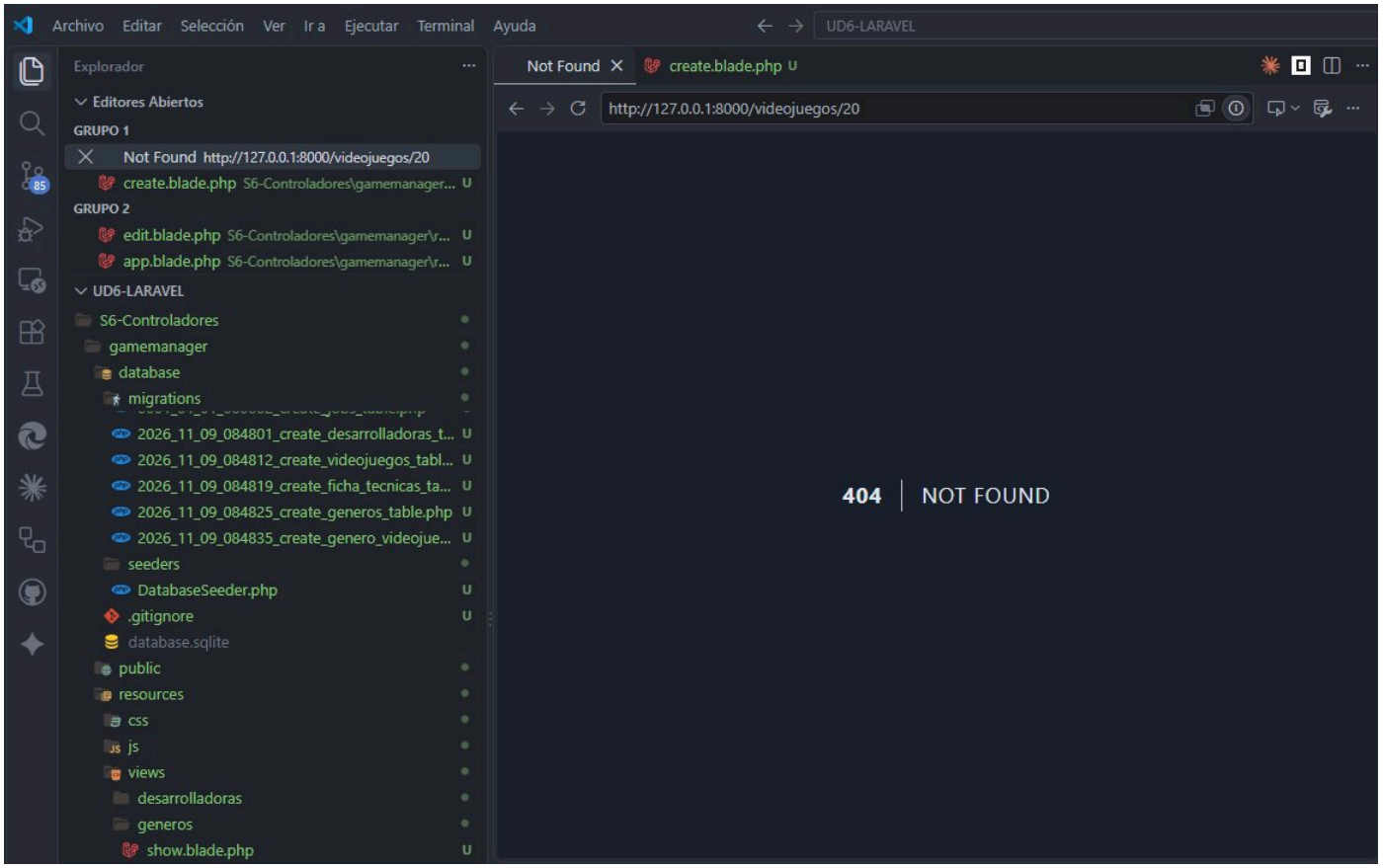


12. Route Model Binding y 404

Captura show(Curso $curso), detalle y error 404 al usar un identificador inexistente.

Palabra clave: Route Model Binding

Comentario: Comprueba la inyección de dependencias por Route Model Binding en el método show(Videojuego $videojuego), lanzando un error 404 automático al solicitar un identificador no registrado

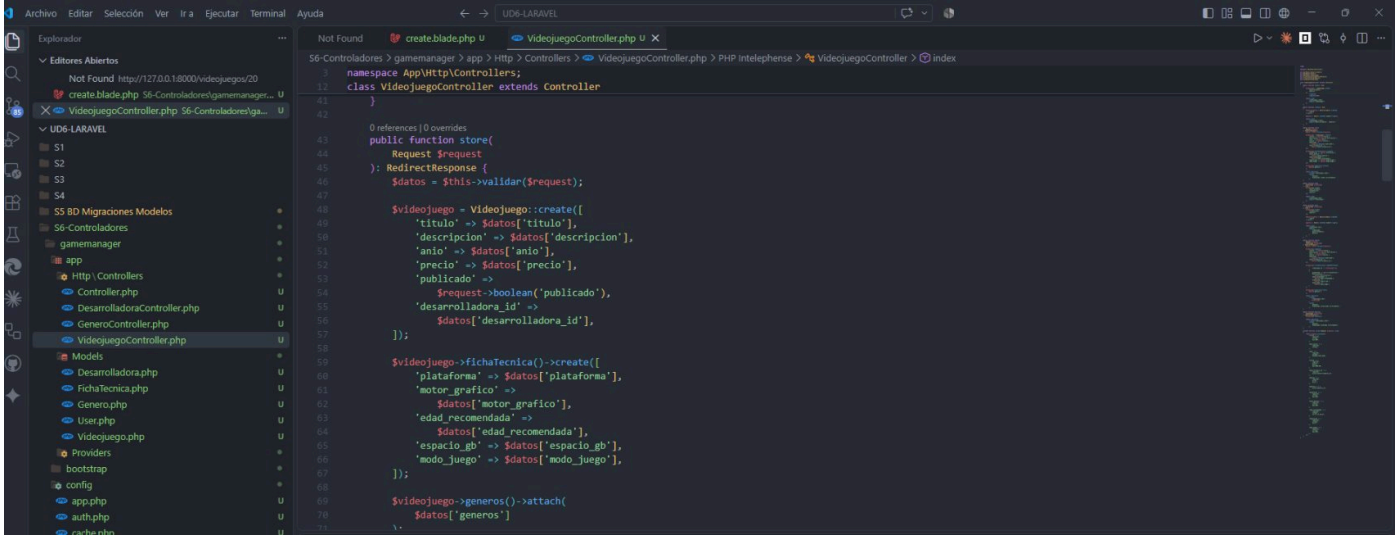


13. Store Form Request

Muestra authorize(), rules(), messages() y validated() en store().

Palabra clave: StoreCursoRequest

Comentario: Definición de las reglas de validación en la recepción de peticiones de alta, comprobando tipos de datos, longitud máxima y la existencia de la clave foránea en la tabla desarrolladoras



```php
namespace App\Http\Controllers;
class VideojuegoController extends Controller
    }

    0 references | 0 overrides
    public function store(
        Request $request
    ): RedirectResponse {
        $datos = $this->validar($request);

        $videojuego = Videojuego::create([
            'titulo' => $datos['titulo'],
            'descripcion' => $datos['descripcion'],
            'anio' => $datos['anio'],
            'precio' => $datos['precio'],
            'publicado' =>
                $request->boolean('publicado'),
            'desarrolladora_id' =>
                $datos['desarrolladora_id'],
        ]);

        $videojuego->fichaTecnica()->create([
            'plataforma' => $datos['plataforma'],
            'motor_grafico' =>
                $datos['motor_grafico'],
            'edad_recomendada' =>
                $datos['edad_recomendada'],
            'espacio_gb' => $datos['espacio_gb'],
            'modo_juego' => $datos['modo_juego'],
        ]);

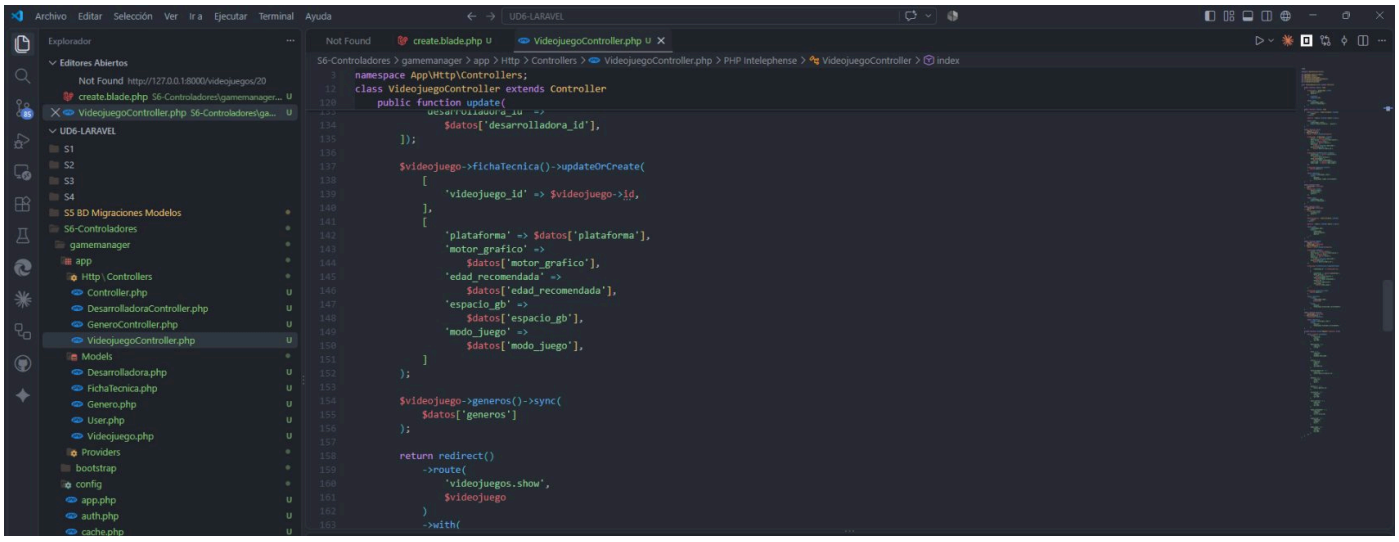
        $videojuego->generos()->attach(
            $datos['generos']
        );
```

14. Update Form Request y unique

Muestra UpdateCursoRequest con regla unique que ignore el registro actual.

Palabra clave: Rule::unique()->ignore()

Comentario: Muestra la validación de los datos actualizados y la sincronización limpia de las relaciones muchos a muchos evitando inconsistencias o registros huérfanos



```php
namespace App\Http\Controllers;
class VideojuegoController extends Controller
    public function update(
                $datos['desarrolladora_id'],
        ]);

        $videojuego->fichaTecnica()->updateOrCreate(
            [
                'videojuego_id' => $videojuego->id,
            ],
            [
                'plataforma' => $datos['plataforma'],
                'motor_grafico' =>
                    $datos['motor_grafico'],
                'edad_recomendada' =>
                    $datos['edad_recomendada'],
                'espacio_gb' =>
                    $datos['espacio_gb'],
                'modo_juego' =>
                    $datos['modo_juego'],
            ]
        );

        $videojuego->generos()->sync(
            $datos['generos']
        );

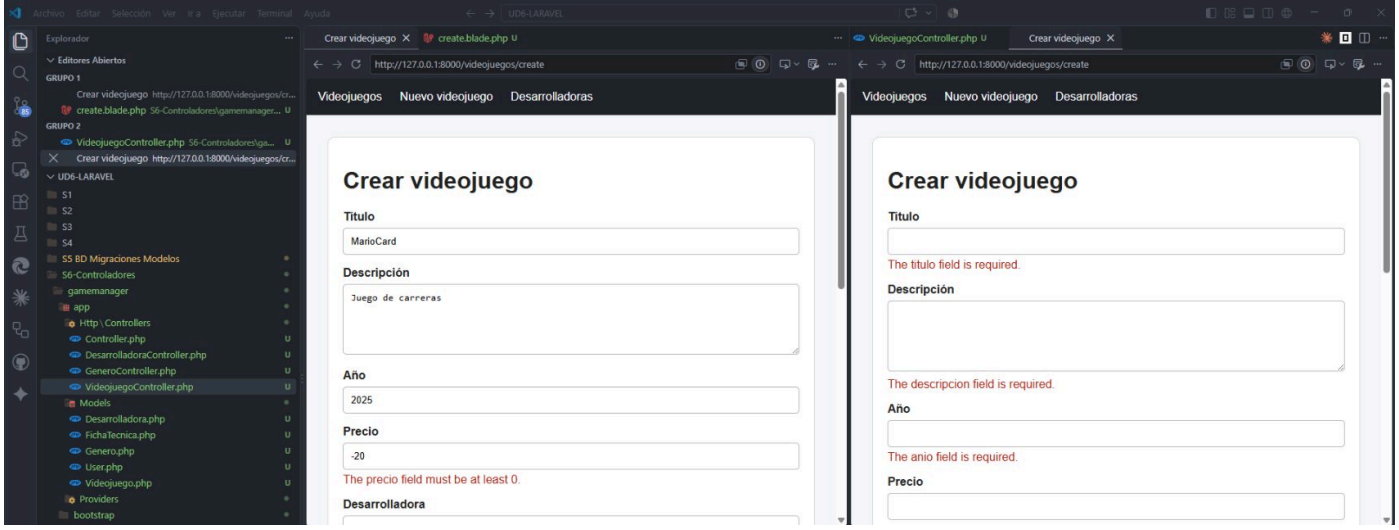
        return redirect()
            ->route(
                'videojuegos.show',
                $videojuego
            )
            ->with(
```

15. Errores, old() y mensajes

Captura un envío inválido con errores personalizados y persistencia de valores.

Palabra clave: old() y @error

Comentario: Comprueba la respuesta de validación en la interfaz: se muestran los mensajes de error asociados a cada campo mediante @error y se conserva el texto introducido con la función auxiliar old()

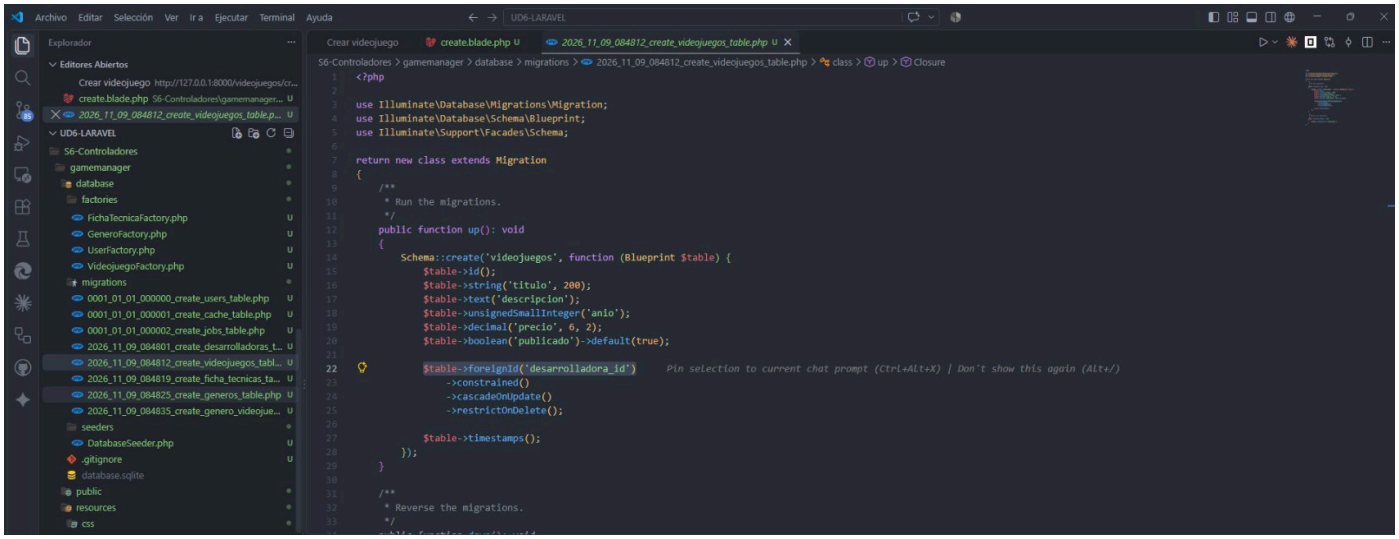


16. Migración y clave foránea

Crea el modelo Leccion, tabla lecciones y la clave foránea curso_id.

Palabra clave: foreignId()->constrained()

Comentario: Migración donde se declara la columna desarrolladora_id como clave foránea vinculada a la tabla padre con actualización en cascada y restricción de borrado para proteger la integridad relacional



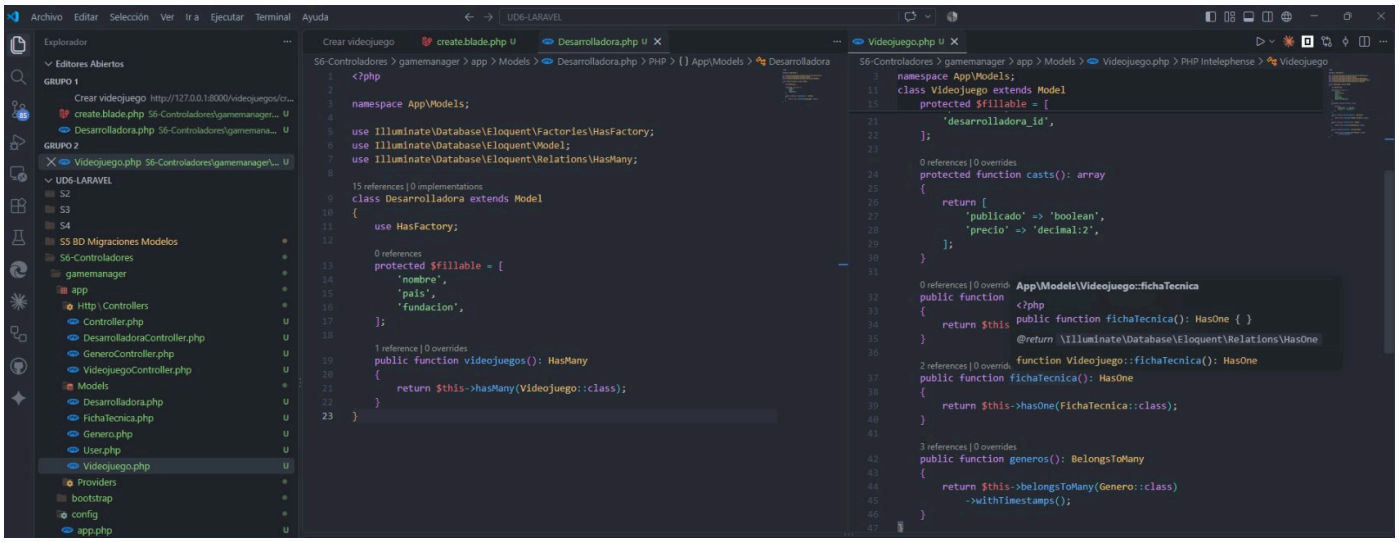
17. Relaciones Eloquent en modelos

Muestra Curso::lecciones() con hasMany y Leccion::curso() con belongsTo.

Palabra clave: hasMany() y belongsTo()

Comentario: Definición de los métodos de relación Eloquent: hasMany en el modelo principal Desarrolladora y belongsTo en el modelo secundario Videojuego con sus

tipos de retorno estrictos

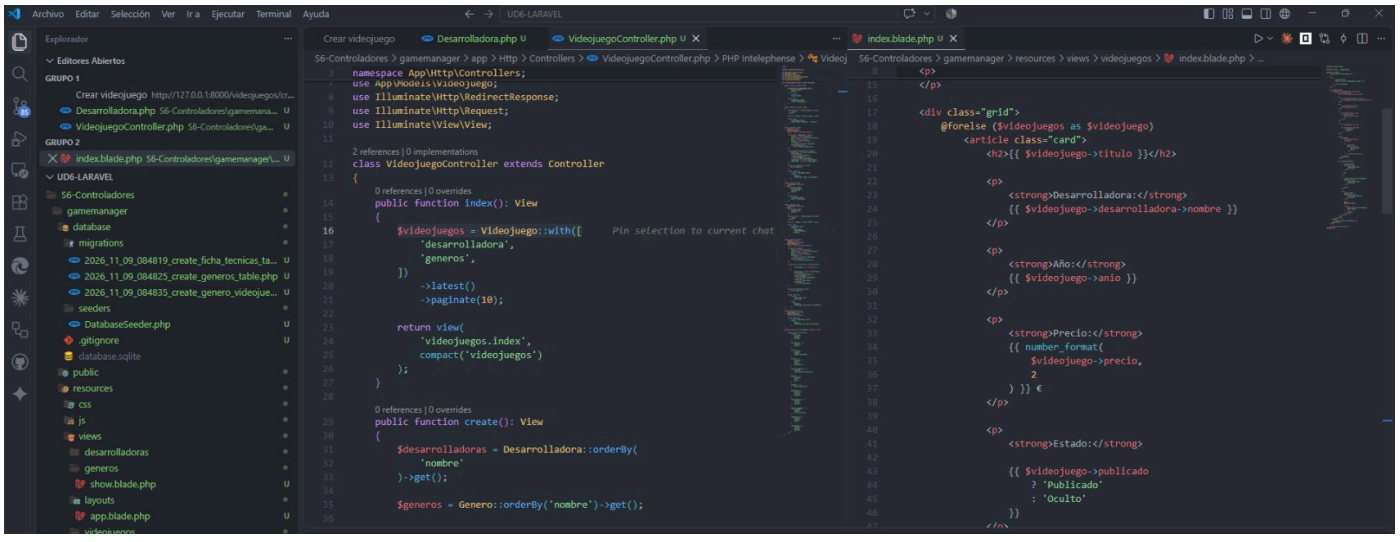


18. Consulta y visualización de relación

Muestra Curso::with('lecciones') y la vista con lecciones de cada curso.

Palabra clave: with()

Comentario: Demuestra el uso de carga ansiosa (Eager Loading) mediante with() en el controlador para precargar las desarrolladoras y evitar el problema de rendimiento N+1 en la vista Blade

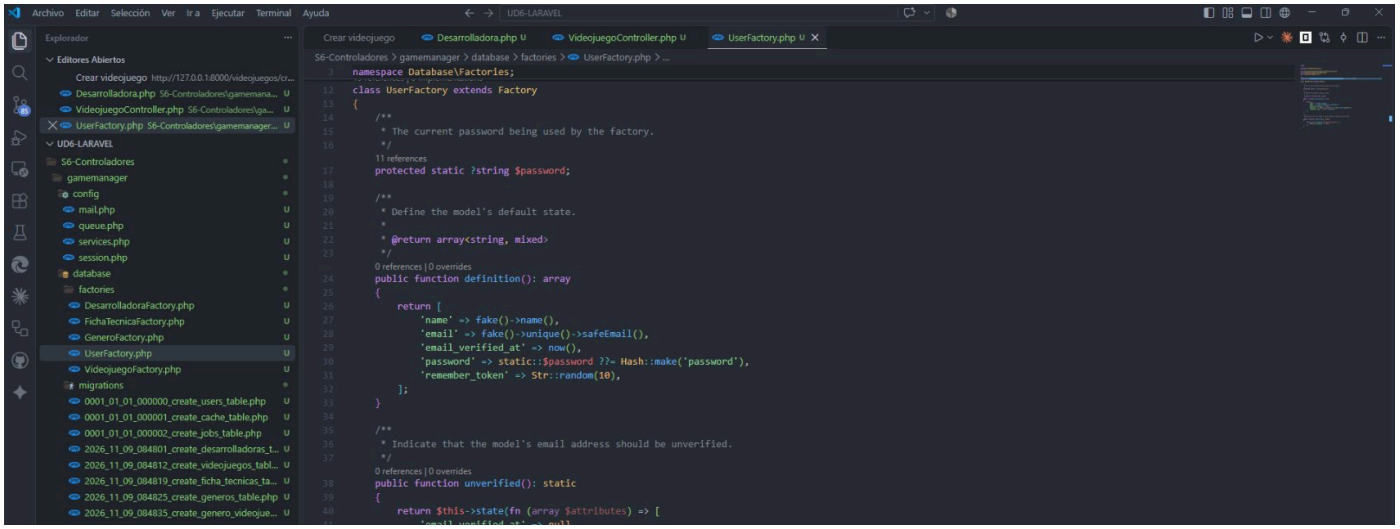


19. Factory y Faker

Captura la factory con definition() y registros de prueba realistas.

Palabra clave: HasFactory / fake()

Comentario: Implementación de la factory utilizando Faker para generar títulos, descripciones, años, precios y la asociación automática a Desarrolladora::factory()

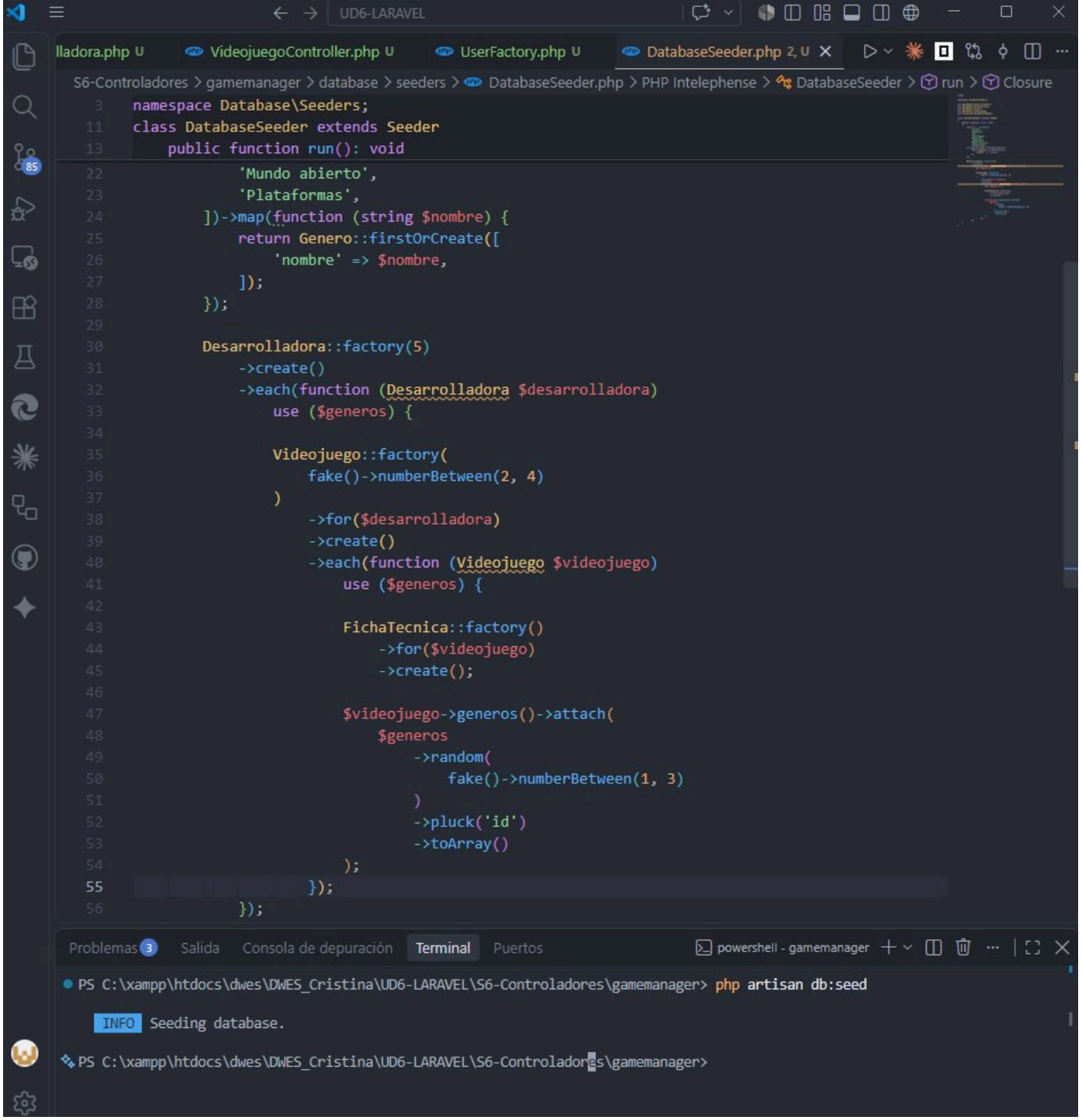


20. Seeder y comprobación final

Captura DatabaseSeeder con factories, db:seed y la aplicación mostrando datos relacionados.

Palabra clave: php artisan db:seed

Comentario: Código del seeder orquestando la creación encadenada de desarrolladoras, videojuegos, fichas técnicas y géneros mediante factories y mostramos prueba por terminal.



Reflexión final

He aprendido la separación de responsabilidades que promueve MVC, gestionando la lógica de negocio en modelos y controladores mientras Blade se encarga exclusivamente de la representación. He comprendido la importancia de Eloquent para las relacion y como el "with" previene problemas de rendimiento. Me ha quedado mucho más claro el uso de las factories y seeders para la creación de entonos de pruebas

Problemas y soluciones: Al actualizar a PHP 8.5 en el entorno de desarrollo local con SQLite, surgieron avisos de obsolescencia (Deprecated: Constant

PDO::MYSQL_ATTR_SSL_CA) procedentes de la configuración residual de MySQL. Lo resolví actualizando la sintaxis a Pdo\Mysql::ATTR_SSL_CA en config/database.php y limpiando la caché con php artisan config:clear. Use también restrictOnDelete() para impedir que se eliminen desarrolladoras con videojuegos asociados.

Archivos destacados: app/Models/Videojuego.php, app/Models/Desarrolladora.php, app/Models/FichaTecnica.php, app/Models/Genero.php, app/Http/Controllers/VideojuegoController.php, app/Http/Controllers/DesarrolladoraController.php, routes/web.php, database/seeders/DatabaseSeeder.php, resources/views/layouts/app.blade.php, resources/views/videojuegos/_form.blade.php